Rising Water Project

FLOOD PREDICTION USING MACHINE LEARNING

A Machine Learning-Based Web Application for Flood Risk Prediction Using Weather and Rainfall Data

PROJECT DOCUMENTATION

Submitted by

Raviteja Tulasi

Kodali Shamily

Kalari Lahari

Course: Artificial intelligence & Machine learning

B.Tech – Computer Science and Engineering (Artificial Intelligence)

KKR & KSR Institute of Technology and Sciences

Academic Year 2025–2026

DECLARATION

We hereby declare that the project entitled “Flood Prediction Using Machine Learning” is an original academic work carried out as part of our learning and project development activities. This report presents the analysis, design, machine learning methodology, implementation, testing and deployment of the proposed flood prediction system.

ACKNOWLEDGEMENT

We express our sincere gratitude to the faculty and management of KKR & KSR Institute of Technology and Sciences for providing the academic environment, guidance and resources required to complete this project. We also acknowledge the open-source technologies used in the development of the system, including Python, Flask, Scikit-learn, Pandas, NumPy, XGBoost, GitHub and Render.

ABSTRACT

Floods are among the most destructive natural disasters and can cause serious damage to human life, agriculture, infrastructure and the economy. This project presents a Machine Learning-based Flood Prediction System that analyzes weather and rainfall parameters to estimate flood risk. The system uses ten input features and compares Logistic Regression, Decision Tree, Random Forest and XGBoost. Random Forest was selected for deployment after achieving 95.65% accuracy in the project evaluation. The model is integrated into a Flask web application with registration, login, session management, a personalized dashboard, flood probability estimation and prediction history.

TABLE OF CONTENTS

CHAPTER 1 – INTRODUCTION

1.1 Introduction

Flood prediction is an important application of data science and machine learning. Weather and rainfall parameters contain patterns that may be associated with flood occurrence. This project develops an end-to-end system that accepts environmental inputs, processes them using a trained classification model and presents the predicted flood class and estimated probability through an interactive web interface.

1.2 Problem Statement

Manual analysis of historical weather and rainfall datasets is time-consuming and difficult for non-technical users. A software system is required to automate the analysis of selected environmental features and provide a consistent machine-learning-based flood-risk classification.

1.3 Objectives

- Prepare historical weather and rainfall data.
- Train and compare multiple classification algorithms.
- Select and deploy a suitable model.
- Develop a Flask web application.
- Provide classification and flood probability.
- Implement authentication and user sessions.
- Store user-specific prediction history.
- Deploy the system online.

1.4 Scope

The project focuses on binary flood classification using ten features available in the training dataset. It demonstrates the integration of machine learning, web development and database management.

1.5 Limitations

- Prediction quality depends on the historical dataset.
- No automatic live weather, river-level, terrain or satellite data is used.
- The dataset is class-imbalanced.
- The result is not an official emergency warning.

CHAPTER 2 – LITERATURE SURVEY

2.1 Traditional Approaches

Traditional flood forecasting uses rainfall-runoff models, hydrological simulation, river discharge measurements, water-level gauges and threshold-based warning systems.

2.2 Machine Learning Approaches

Machine learning can identify patterns in historical environmental data. Logistic Regression provides a linear baseline, Decision Trees learn nonlinear rules, Random Forest combines multiple trees, and XGBoost applies gradient boosting.

2.3 Research Gap

Many academic projects stop after notebook-based model evaluation. This project extends the workflow into a complete web application with authentication, probability output, dashboard statistics, history and cloud deployment.

CHAPTER 3 – SYSTEM ANALYSIS

3.1 Existing System

A basic manual workflow requires users to inspect spreadsheets or execute notebook code. It does not naturally provide authentication, dashboards or persistent prediction records.

3.2 Proposed System

The proposed system integrates a trained Random Forest classifier with Flask. Authenticated users submit ten input values, receive a classification and probability, and review stored prediction records.

3.3 Functional Requirements

- Registration and login.
- Session management and logout.
- Ten-feature prediction form.
- Flood/no-flood classification.
- Flood probability calculation.
- Prediction storage.
- Dashboard statistics.
- Prediction history.

3.4 Non-Functional Requirements

- Usability and clear navigation.
- Reliable feature ordering and model loading.
- Secure authenticated access.
- Maintainable project structure.
- Local and cloud portability.

CHAPTER 4 – SYSTEM DESIGN

4.1 Architecture

User → Web Interface → Flask Application → Machine Learning Model → Prediction Result → Database → Dashboard and History.

4.2 Modules

The system contains User Management, Weather Data, ML Model, Prediction Result, Dashboard and Prediction History modules.

4.3 ER Design

The conceptual design contains Users, Weather_Data, ML_Model and Prediction_Result entities. A user can create multiple weather records and predictions. Each prediction is associated with input data and the model used.

4.4 Workflow

A user registers, logs in, enters weather and rainfall data, submits the prediction form, receives the model result and probability, and can later review the stored result through the dashboard and history page.

CHAPTER 5 – DATASET AND DATA PREPROCESSING

5.1 Dataset

The final feature matrix contains ten input columns and a binary target column named flood.

5.2 Target

The target uses 0 for No Flood and 1 for Flood.

5.3 Class Distribution

The development dataset contained 115 records: 99 No Flood and 16 Flood. This imbalance should be considered when interpreting performance.

5.4 Preprocessing

The workflow includes loading data, inspecting columns and missing values, separating features and target, splitting training and testing data, training models, evaluating results and saving the selected model.

CHAPTER 6 – MACHINE LEARNING METHODOLOGY

6.1 Algorithms

Logistic Regression, Decision Tree, Random Forest and XGBoost were evaluated.

6.2 Performance

The reported accuracies were 91.30% for Logistic Regression, 95.65% for Decision Tree, 95.65% for Random Forest and 86.96% for XGBoost.

6.3 Selected Model

Random Forest was selected for deployment. The saved model corresponds to RandomForestClassifier(random_state=42).

6.4 Confusion Matrix

The reported confusion matrix was [[20, 0], [1, 2]], meaning 20 no-flood examples and two flood examples were correctly classified, while one flood example was missed.

6.5 Probability and Serialization

The application displays the model probability for the flood class. The final model is serialized with Joblib as models/floods.save.

CHAPTER 7 – SYSTEM IMPLEMENTATION

7.1 Flask Backend

Flask handles URL routing, request processing, template rendering, sessions, model integration and database interaction.

7.2 Authentication

Registration creates user accounts, while login authenticates users before access to protected dashboard, prediction and history functions.

7.3 Prediction

The form collects ten values. The backend creates the feature input in training order, calls the saved model, calculates flood probability, stores the result and renders the appropriate result page.

7.4 Dashboard and History

The dashboard summarizes total, flood and no-flood predictions. The history page displays previous predictions with input values, model information, probability, result and date.

CHAPTER 8 – RESULTS AND APPLICATION SCREENS

8.1 Home Page

Introduces the project and prediction process.

8.2 Login and Registration

Provide account creation and authenticated access.

8.3 Dashboard

Displays personalized statistics, quick actions and recent predictions.

8.4 Prediction Page

Collects all ten weather and rainfall input values.

8.5 Result Pages

Display either Flood Risk or No Flood together with probability and relevant information.

8.6 Prediction History

Displays previous user-specific predictions and complete input information.

CHAPTER 9 – TESTING

9.1 Testing Strategy

Functional testing verifies registration, login, protected access, prediction processing, probability calculation, dashboard updates, history display and logout.

9.2 Model Testing

A known high-risk sample generated Flood Risk with a reported probability of 99.00%. The selected Random Forest achieved approximately 95.65% accuracy on the reported test split.

9.3 Evaluation Considerations

Future evaluation should include precision, recall, F1-score, ROC-AUC and cross-validation, especially because the flood class is the minority class.

CHAPTER 10 – DEPLOYMENT

10.1 Deployment

The project is version-controlled using Git, hosted on GitHub and deployed as a web service. Dependencies are installed from requirements.txt and the Flask application is started using a production WSGI server.

10.2 Considerations

Production deployment requires compatible Python dependencies, correct model paths, environment-based secrets and persistent database storage when long-term records are required.

CHAPTER 11 – ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

11.1 Advantages

- End-to-end machine learning and web integration.
- Simple prediction interface.
- Probability and classification output.
- Personalized dashboard and history.
- Cloud deployment.

11.2 Limitations

- Small and imbalanced dataset.
- No live weather or geospatial integration.
- Predictions are limited to learned historical patterns.
- Educational prototype rather than an official warning system.

11.3 Future Scope

- Live weather API integration.
- Location-based prediction and maps.
- River-level, soil-moisture and satellite data.
- Larger datasets and advanced validation.
- Email/SMS alerts.
- Admin dashboard and downloadable reports.
- Time-series and deep-learning approaches.

CHAPTER 12 – CONCLUSION

12.1 Conclusion

The project demonstrates the complete lifecycle of a machine learning application: dataset preparation, model training, algorithm comparison, model selection, serialization, backend integration, authentication, prediction tracking and deployment. Random Forest achieved 95.65% accuracy in the reported evaluation and was selected for deployment. The final application allows authenticated users to submit ten weather and rainfall features and receive a flood/no-flood classification with estimated probability. Dashboard and history modules extend the work beyond a standalone notebook into a usable full-stack machine learning system.

DATASET FEATURES

No.	Feature	Description
1	Temp	Temperature
2	Humidity	Relative humidity
3	Cloud Cover	Cloud cover percentage
4	ANNUAL	Annual rainfall
5	Jan-Feb	January–February rainfall
6	Mar-May	March–May rainfall
7	Jun-Sep	June–September rainfall
8	Oct-Dec	October–December rainfall
9	avgjune	Average June rainfall
10	sub	Additional rainfall feature

MODEL PERFORMANCE

Model	Accuracy
Logistic Regression	91.30%
Decision Tree	95.65%
Random Forest	95.65%
XGBoost	86.96%

SAMPLE FLOOD-RISK INPUT

Feature	Value
Temperature	30
Humidity	71
Cloud Cover	41
Annual Rainfall	4226.4
Jan-Feb	22.2
Mar-May	363.0
Jun-Sep	3451.3
Oct-Dec	389.9
Average June Rainfall	337.233333
Sub Rainfall	826.3

For the saved model used during testing, this sample produced a Flood Risk classification with a reported flood probability of 99.00%.

TEST CASES

Test ID	Test Case	Expected Result
T01	Register with valid details	Account created
T02	Login with valid credentials	Dashboard displayed
T03	Submit ten valid inputs	Prediction generated
T04	Known high-risk sample	Flood Risk displayed
T05	Low-risk sample	No Flood displayed
T06	Open dashboard	Statistics updated
T07	Open history	Stored predictions displayed
T08	Logout	Session ended

REFERENCES

Scikit-learn Documentation.

Flask Documentation.

Pandas Documentation.

NumPy Documentation.

XGBoost Documentation.

Joblib Documentation.

GitHub Documentation.

Render Documentation.

APPENDIX – DISCLAIMER

This application is developed for educational and research purposes. Predictions are generated from a machine learning model trained on historical data and user-provided values. The output must not be treated as an official flood warning, emergency alert or substitute for information issued by meteorological departments, disaster-management agencies or local authorities.